

Klassiker 23: Top-Seller-Report

Zum Foliensatz 23 — SQLite Vertiefung · SoSe 2026

Dorian Zwanzig

Die Aufgabe

Der Abschluss-Klassiker der Datenbank-Phase: Beantworte für den Veggie-Soles-Shop mit **einer** SQL-Abfrage die Frage „**Welches Produkt macht den meisten Umsatz?**“ Baue eine In-Memory-Datenbank (`sqlite3.connect(":memory:")`) mit den Tabellen unten und gib den Report **absteigend nach Umsatz** aus. Zeile 1 ist der Top-Seller — das *teuerste* Produkt muss es nicht sein: die **Menge** entscheidet.

Tabelle produkt

produkt_id	name	preis
1	Eco-Sneaker	89.95
2	Hemp-High	109.00
3	Bambus-Boot	135.50

Tabelle bestellung

bestell_id	kunde	produkt_id	menge
1	Anna	1	2
2	Max	2	1
3	Anna	3	1
4	Max	1	3
5	Anna	2	2
6	Max	3	1

Erwartetes Verhalten

Top-Seller-Report Veggie Soles

```
-----  
Eco-Sneaker    5 Stk    449.75 EUR  
Hemp-High      3 Stk    327.00 EUR  
Bambus-Boot    2 Stk    271.00 EUR
```

Top-Seller: Eco-Sneaker (449.75 EUR)

Erlaubte Bausteine

Alles bis Notebook 23 — insbesondere: `sqlite3` (`connect`, `execute`, `executemany`, `fetchall`) · `CREATE TABLE` / `INSERT` · `JOIN ... ON` · `SUM(...)`, `ROUND(...)` · `AS`-Alias · `GROUP BY` · `ORDER BY ... DESC` · f-Strings zum Formatieren

Gestufte Hinweise

Erst selbst probieren — dann Hinweis für Hinweis aufdecken.

Hinweis 1 (Ansatz): Baue zuerst nur den `JOIN` über `produkt_id` und sieh dir das Ergebnis mit `SELECT *` an: jede Bestellzeile kennt jetzt ihren Preis.

Hinweis 2 (der Kern): Der Umsatz ist `preis * menge` — **pro Zeile, vor dem Gruppieren**. Erst danach wird pro Produkt aufsummiert.

Hinweis 3 (Struktur): Das Gerüst der fertigen Abfrage:

```
SELECT produkt.name, SUM(menge), SUM(preis * menge) AS umsatz  
FROM bestellung JOIN produkt ON ...  
GROUP BY ...  
ORDER BY umsatz DESC
```

Bonus

Umsatz pro Kunde: `GROUP BY bestellung.kunde` — wer kauft mehr, Anna oder Max? **Durchschnittsbestellwert:** was liefert `AVG(preis * menge)`? *Musterlösung nach der Vorlesung: 90_klassiker_topseller.py im Foliensatz-Ordner 23.*